代码部分：

```python
import math
import random

# 计算最大公约数
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

# Miller-Rabin 素性测试，判断 n 是否为质数
def is_prime(n, k=5):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0:
        return False
    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1
    for _ in range(k):
        a = random.randint(2, n - 2)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(s - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True

# Pollard's Rho 算法的迭代过程，返回因子和迭代次数
def pollards_rho_iter(n, f):
    if n == 1:
        return (1, 0)
    x = random.randint(1, n - 1)
    y = x
    d = 1
    iterations = 0
    while d == 1:
        x = f(x, n)
        y = f(f(y, n), n)
        d = gcd(abs(x - y), n)
        iterations += 1
    return (d, iterations) if d != n else (None, iterations)

# 定义迭代函数 f(x) = (x^2 + 1) % n
def f(x, n):
    return (x ** 2 + 1) % n

# 使用 Pollard's Rho 算法分解 n，返回分解出的一个因子和迭代次数
def pollards_rho(n):
    if n == 1:
        return (1, 0)
    if is_prime(n):
        return (n, 1)
    divisor, iterations = pollards_rho_iter(n, f)
    while divisor is None or divisor == 1 or divisor == n:
        divisor, new_iterations = pollards_rho_iter(n, f)
        iterations += new_iterations
    return (divisor, iterations)

# 分解整数并输出结果及迭代次数
def factor_and_count_iterations(n):
    print(f"正在分解整数 {n}...")
    if n == 1:
        print("1 无需分解")
        return 0
    if is_prime(n):
        print(f"{n} 是质数，无法分解为更小的非平凡因子")
        return 1
    divisor, iterations = pollards_rho(n)
    other_factor = n // divisor
    print(f"分解结果：{n} = {divisor} * {other_factor}")
    print(f"分解 {n} 共经历 {iterations} 次迭代")
    return iterations

if __name__ == "__main__":
    numbers = [262063, 9420457, 181937053]
    for num in numbers:
        total_iterations = factor_and_count_iterations(num)
        print(f"分解整数 {num} 总共使用 {total_iterations} 次迭代\n")
```

3次运行结果：

```
正在分解整数 262063...
分解结果：262063 = 521 * 503
分解 262063 共经历 19 次迭代
分解整数 262063 总共使用 19 次迭代

正在分解整数 9420457...
分解结果：9420457 = 2351 * 4007
分解 9420457 共经历 25 次迭代
分解整数 9420457 总共使用 25 次迭代

正在分解整数 181937053...
分解结果：181937053 = 12391 * 14683
分解 181937053 共经历 79 次迭代
分解整数 181937053 总共使用 79 次迭代
```

```
正在分解整数 262063...
分解结果：262063 = 503 * 521
分解 262063 共经历 7 次迭代
分解整数 262063 总共使用 7 次迭代

正在分解整数 9420457...
分解结果：9420457 = 2351 * 4007
分解 9420457 共经历 25 次迭代
分解整数 9420457 总共使用 25 次迭代

正在分解整数 181937053...
分解结果：181937053 = 12391 * 14683
分解 181937053 共经历 29 次迭代
分解整数 181937053 总共使用 29 次迭代
```

```
正在分解整数 262063...
分解结果：262063 = 503 * 521
分解 262063 共经历 7 次迭代
分解整数 262063 总共使用 7 次迭代

正在分解整数 9420457...
分解结果：9420457 = 2351 * 4007
分解 9420457 共经历 50 次迭代
分解整数 9420457 总共使用 50 次迭代

正在分解整数 181937053...
分解结果：181937053 = 14683 * 12391
分解 181937053 共经历 63 次迭代
分解整数 181937053 总共使用 63 次迭代
```